

Riptide Unified Rights Platform

From Independent Publisher to SaaS Platform
to Ecosystem Music Company

Draft Memo – Prepared on 12/5/25 by Math Team

1. Executive Summary

Riptide is at an inflection point.

The company has developed real, working infrastructure – catalog systems, royalty workflows, CWR delivery, a client portal, and a Tipalti-enabled payout system; significantly beyond what most independent publishers possess. However, the platform needs to elevate further to become a lucrative, public-facing product.

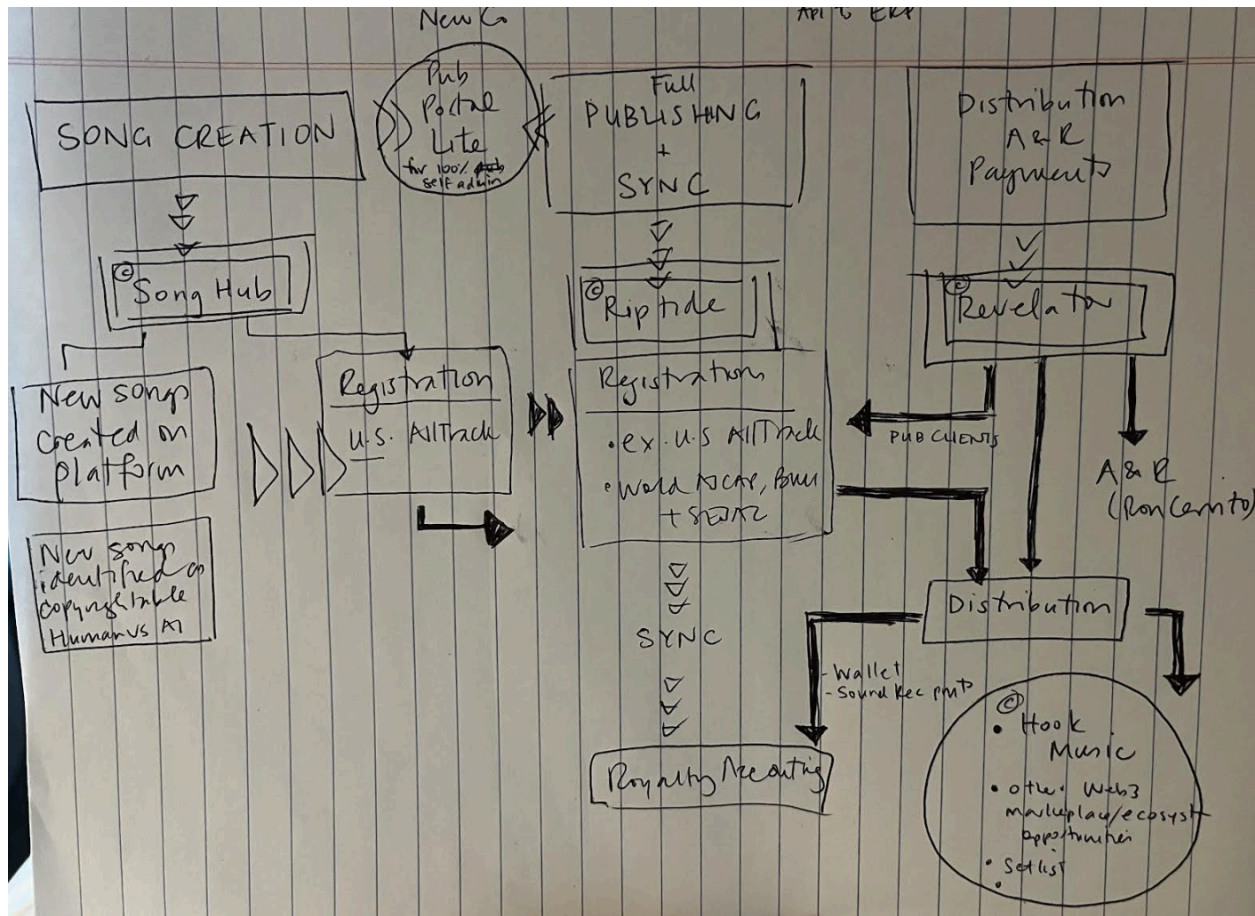
At the same time, Riptide has unique relationships with companies like Revelator, Switchchord, SongHub, and Hook Music that together form the nucleus of a broader Ecosystem Music Company.

This memo outlines:

- 1.) **A kernel design:** the rights + money core that Riptide must own
- 2.) **A partner strategy:** who plugs into the kernel and how
- 3.) **A clear path** from today > MVP > V1 > multi-company ecosystem
- 4.) **A mid-term revenue strategy:** how the platform becomes a sustainable, scalable business

This is the positioning Riptide needs in order to evolve from an independent publisher with custom tools into a platform company powering a network of complementary music businesses.

2. The North Star



Riptide's North Star is to become the rights and revenue kernel for the broader music ecosystem. A central piece of infrastructure that does the following:

- Receives contracts
- Resolves identity
- Maps agreements to works
- Performs registrations (CWR)
- Ingests usage and earnings
- Applies splits and recoupment

- Drives business intelligence dashboards
- Orchestrates payouts

Partners like Switchchord, SongHub, Hook, and Revelator supply creators, content, and data all in an application layer.

In turn, Riptide supplies the core infrastructure they rely on.

This is the foundation of the *Ecosystem Music Company* vision.

3. The Kernel Design

The kernel is the heart of the Unified Rights Platform (URP). It defines how data flows, how money flows, and how partners interoperate.

It is fully partner-agnostic in which partners provide data, but Riptide defines the data model, logic, and workflows.

The kernel consists of seven core subsystems, outlined below.

3.1 Portal Lite (Intake & Creator Onboarding)

Portal Lite solves the operational bottlenecks that surfaced in every conversation with Riptide:

- Works come in from multiple pathways (writers, SongHub, Riptide team, legacy catalogs)
- Metadata quality varies widely, which means mismatch is common
- Creators don't consistently provide splits or collaborator info
- Unpredictable manager/publisher approval loops

Portal Lite expands the portal into a multi-company, self-admin intake engine with:

- Identity pre-matching
- Split capture & validation
- Structured approval flows
- Metadata validation
- Conflict resolution queues
- Partner-specific intake configurations
- Support for third-party partners onboarding large volumes of works

Key Takeaway:

Portal Lite dramatically reduces manual work and enables scaled, multi-catalog ingestion.

3.2 Identity Graph

One of the core elements the URP solves is challenges with identity:

- Writer names appear in an array of formats
- Agreements refer to parties differently than statements
- External platforms provide partial or inconsistent data
- Remixes create derivative lineage relationships

Therefore, the identity graph must:

- Dedupe creators
- Support external identity IDs (e.g., Switchchord credentials)
- Link multiple roles (writer, artist, publisher, label)

- Maintain audit trails
- Handle cross-catalog relationships
- Power every downstream workflow

Key Takeaway:

Identity is Riptide's strategic moat. In my opinion, identity must remain a kernel-governed, multi-tenant, auditable subsystem.

3.3 Agreement Module

In the current workflow, contract ingestion is:

- Manual
- Inconsistent
- Error-prone
- Stored in FileMaker in a schema that is hard to decouple

Switchchord becomes the canonical contract ingestion engine, converting legal PDFs into structured JSON (splits, terms, territories, parties, amendments, etc.).

But the kernel owns the legal logic:

- Mapping agreement parties to identity
- Validating splits
- Binding agreements to works & recordings
- Handling amendments
- Establishing canonical versions

Key Takeaway:

Switchchord extracts the contract, while Riptide interprets and governs it.

3.4 Catalog and CWR Engine

Riptide's existing CWR pipeline is already operational, which is a major advantage over other publishers.

However, there are some constraints that need to be considered:

- It needs better validation
- CWR errors are still manually handled
- Multi-tenant catalogs (SongHub, Hook) require bespoke handling
- Version support is mixed

The upgraded engine must:

- Standardized validation
- Structured error queues
- Multi-tenant catalogs (Riptide, SongHub, Hook, external clients)
- Partner-specific configuration
- Mappings for derivatives and remixes
- Inbound corrections from societies

Key Takeaway:

This becomes the registration router connecting partners to the world's rights organizations.

3.5 Statement Ingestion (Usage > Earnings > Ownership)

Current state:

- Statements vary wildly in structure
- Mapping to works can fail without better identity linkage
- Multi-type revenue handling (mechanical, performance, master use, UGC)
- Advances & recoupment are tracked inconsistently

Partner inputs:

- **Revelator** provides structured usage and earnings
- **Hook** will provide remix performance and revenue
- **SongHub** may send publishing earnings to Riptide for global admin

Kernel responsibilities:

- Ingestion
- Normalization
- Mapping to identity and catalog
- Recoupment logic
- Validation and reconciliation
- reporting

Key Takeaway:

This is the financial engine that drives BI and payouts.

3.6 Business Intelligence Semantic Layer

Riptide's internal workflows currently rely on:

- Spreadsheets
- Manual exports
- Inconsistent internal reports

Partners consistently ask for better visibility.

In turn, the BI layer must provide:

- Creator dashboards (earnings, geography, sources)
- Publisher dashboards
- Multi-tenant partner dashboards
- Riptide executive dashboards
- CSV/Excel download
- Unified semantic model

Key Takeaway:

This becomes the insights moat for the entire ecosystem.

3.7 Payout Engine (Kernel Logic > Tipalti Execution)

Current state:

- Riptide uses Tipalti well, but all payout logic is outside of it
- Payment timing is often manual

- Reporting requires multiple exports
- General ledger integration and sync is still future-state

Kernel handles:

- Payout logic
- Thresholds
- Scheduling
- Exceptions
- General Ledger preparation

Tipalti handles:

- KYC
- AML
- Taxes
- Global execution

Key Takeaway:

Riptide orchestrates payout logic; Tipalti executes it.

4. Partner Strategy (Who Plugs Into the Kernel & How)

Riptide's objective here is not to build every capability, it's really more about orchestrating an ecosystem of highly specialized (even hand-picked) partners.

Below is the optimized distribution of responsibilities.

4.1 Switchchord – Contract Ingestion & Legal Data Extraction

Switchchord's Strengths:

- Bulk contract parsing
- AI-assisted legal interpretation
- Structured JSON output
- Working with major catalogs (Primary Wave, Concord)
- Accelerating deal onboarding

They do not own identity. They assist identity by supplying reliable contract metadata.

Kernel responsibilities:

- Identity resolution
- Relationship modeling
- Agreement linking
- Legal lineage
- Versioning

Key Takeaway:

Switchchord extracts the contract; Riptide governs identity, rights, lineage, and logic.

4.2 SongHub: Creator Intake & US Registrations

SongHub has:

- Thousands of creators
- Direct connections with educational institutions
- Integration with AllTracks for US registrations

SongHub's incentive:

- Avoid building publishing infrastructure
- Scale creator onboarding
- Offer publishing services to their users

Riptide's opportunity:

- Become SongHub's global publishing backbone
- Dramatically expand catalog volume
- Drive multi-tenant royalties & payouts

Key Takeaway:

SongHub is a creator funnel.

Riptide is the infrastructure they deploy on.

4.3 Hook Music: Remix & UGC Monetization

Hook provides:

- Remix/UGC generation
- High-growth UGC content
- Creator activity

Hook's incentive:

- Use Riptide as royalty and payment infrastructure
- Monetize remixes legally and profitably
- Share revenue with Riptide

Riptide's opportunity:

- Become the trusted payout engine for remix culture
- Own the lineage metadata for derivative works
- Unlock new revenue streams

Key Takeaway:

Hook is a value-add creativity partner.

Riptide is the rights and royalty engine for their output.

4.4 Revelator: Distribution & Real-Time Data

Revelator provides:

- Distribution rails
- Normalized usage
- Real-time analytics
- Advanced blockchain-native workflows
- Deep founder alignment with Riptide

Revelator's incentive:

- Co-build publishing

- Feed high-quality usage to Riptide
- Leverage Riptide's rights logic

Riptide's opportunity:

- Leverage their distribution stack instead of rebuilding
- Ingest high-quality usage
- Co-develop much of the platform

Key Takeaway:

Revelator reduces URP's need to build its own distribution and master-side analytics, allowing Riptide to stay focused on rights and money.

Revelator is the usage and data source of truth and Riptide operationalizes that data.

4.5 Tipalti: Payout Execution

While not exactly a partner, I think it's important to include them in our planning process.

Tipalti does:

- KYC/AML
- Vendor onboarding
- Tax compliance
- Payout execution

Riptide does:

- Calculate who gets paid

- Assemble payout file
- Reconcile results
- Generate General Ledger entries

Key Takeaway:

Tipalti is the payout rails.

Riptide is the logic behind the payout.

5. The Path: Today > MVP > V1 > Ecosystem

This roadmap outlines how Riptide evolves from an internal publishing toolset into the rights and revenue infrastructure for a multi-company ecosystem. It is designed to be realistic for engineering, legible to partners, and compelling for investors.

5.1 Phase 0 — Today (Foundation)

What Exists Now

Riptide already operates a functioning rights pipeline:

- Operational catalog system (FileMaker)
- Client portal
- CWR delivery
- Manual-to-semi-automated royalty workflows
- Tipalti payout rails

- Early partner integrations
- Deep domain knowledge encoded in current processes

Why This Matters

The good news is that this is not a zero-to-one platform build. Riptide already has a working rights engine, the next step is to formalize, modernize, and abstract it into a scalable kernel.

5.2 Phase 1 – MVP (4–6 months)

A tightly scoped MVP proving the full lifecycle across at least two partner catalogs (Riptide + Revelator, ideally). This phase builds the kernel's foundation and tests interoperability early.

Phase 1's Build Scope

Portal Lite > Identity > Agreement > CWR > Statement > BI > Payout

This scope is intentionally high-signal, low-scope: one clean pass through the entire chain, end-to-end.

Expected Developer Deliverables

- Canonical Schemas v1.0 (Identity, Agreement, Work, Recording, Statement)
- Partner Adapters for Switchchord, SongHub, Revelator
- Identity-First Kernel Architecture
- Multi-Tenant Readiness (Basic)
- Event Logging & Data Lineage v0.1
- CWR Export Engine v1.1 (validation + structured error queues)

- Statement Ingestion Engine v1.0
 - BI Semantics v0.5 (creator + catalog-level dashboards)
 - Payout Logic v1.0 (Tipalti integrated)
-

5.3 Phase 1's Architecture & Migration Plan

A.) FileMaker Coexistence Strategy

- FileMaker remains the internal system of record during MVP.
- A lightweight FM Adapter Layer (temporary software layer that sits between the old FileMaker system and the new URP kernel) exposes read-only data to the kernel. This lets us build and test the new kernel independently while FileMaker keeps Riptide's business running.
- New writes (identity, agreements, statements) happen only in the kernel.

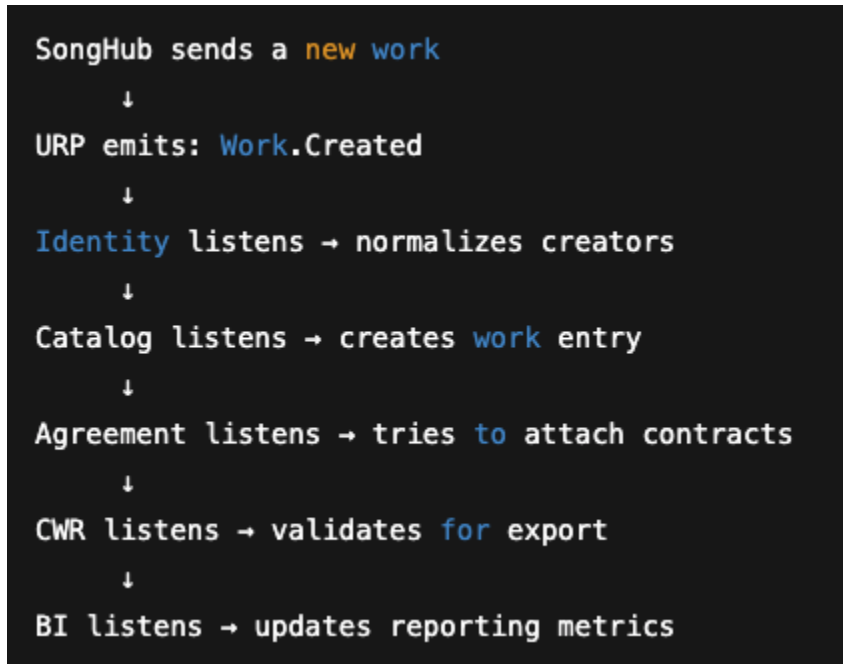
B.) Canonical Schema Migration

- Schemas are defined first.
- Existing FileMaker data is mapped, not migrated, during MVP.
- Migration occurs only when the kernel reaches stability in v1.

C.) Event-Driven Kernel

- Kernel modules communicate by emitting standardized events (Work.Created, Agreement.Updated, Statement.Ingested, Identity.Merged), allowing each subsystem to respond independently, ensuring modularity, scalability, auditability, and partner-friendly interoperability.

A rough visual for example:



- This creates future partner flexibility and auditability.

D.) API-First Internal Architecture

URP is built "API-first," meaning every kernel capability is exposed internally through stable, documented APIs before we ever open anything up to external partners.

Concretely, Phase 1 delivers internal APIs for:

- Identity (create/search/merge entities, manage roles and external IDs)
- Agreements (link contracts to identities and works, retrieve canonical splits)
- Catalog (works, recordings, lineage, catalog membership)
- CWR (validate and queue works for registration, retrieve status and errors)

- Statements (ingest usage/earnings, query normalized statement data)
- Payout Logic (compute entitlements and payout instructions)

These internal APIs are:

- Versioned (v1, v2, etc.) so we can evolve without breaking dependents
- Auth-controlled (service-to-service auth, no partner keys yet)
- Contract-tested (schema and behavior tests in CI)

By forcing all kernel modules and internal tools (Portal Lite, dashboards, partner adapters, etc.) to use these APIs, we ensure that when we decide to sell external API access in Phase 2/3, we are exposing battle-tested, consistent interfaces rather than bolting an API on as an afterthought. This greatly reduces integration risk and makes API monetization a natural extension of how the system already works.

As a result, the MVP proves the kernel works without requiring high-risk migration.

5.4 Phase 1 Objective

Deliver a credible, working interoperability slice that proves URP can manage multiple catalogs and execute the full rights > revenue > payout lifecycle.

5.5 Phase 2 – V1 (6–12 months)

The objective of Phase 2 is to evolve the kernel from MVP into a proper production-grade platform ready for enterprise partners, large catalogs, and investor scaling.

Expected Developer Deliverables

- Canonical Schemas v2.0 (multi-tenant, extensible)
 - Advanced BI (territory insights, predictive analytics baseline)
 - Hook Revenue Ingestion (UGC lineage integration)
 - Partner Dashboards (SongHub, Hook, Revelator)
 - Accounting / GL Integration (QuickBooks, NetSuite)
 - Improved CWR Correction Flow (PRO feedback ingestion)
 - Automated Catalog Ingestion Tools
 - Identity Graph v2.0 (merge history, role linking, external IDs)
 - Internal APIs v1.0 (foundation for later monetized APIs)
-

5.6 Phase 2 Objective

Make URP a credible, enterprise-compatible platform capable of onboarding real partners at scale.

In my opinion, there will be enough meat on the bone here that this is the spot at which Riptide can raise capital, explore strategic partnerships, and begin monetizing early SaaS components.

5.7 Phase 3 – Ecosystem (12–24 months)

Phase 3 is about converting the platform into an ecosystem engine with multi-company functionality.

Expected Developer Deliverables

The kernel now powers:

Core Use Cases

- Riptide publishing
- SongHub creator ecosystem
- Hook UGC/remix economy
- Revelator distribution + analytics
- External partner catalogs

Ecosystem-Level Features

- External APIs v2.0 (Identity, Agreement, Catalog, CWR, Statements, BI, Payout)
- Rights-Data Marketplace Foundations
- Partner Billing and Usage Metering
- Event Versioning and Governance Framework
- Full Multi-Tenant Support (Isolation + Role-Based Access)

5.8 Phase 3 Objective

URP becomes the envisioned multi-company rights and royalty engine, unlocking:

- API monetization
- Data marketplace revenue
- Sync/licensing metadata services

- AI training data licensing
- Catalog valuation tools
- Predictive insights products

If you're wondering what these Phase 3 objectives are all about, head to 6.2 ;)

6. Mid-Term Revenue Strategy

The URP kernel creates several monetization paths. Some are available immediately (SaaS/licensed access), others emerge as the platform matures (API and rights-data marketplace). This section outlines both.

6.1 SaaS Revenue / Licensed Access

Although not the most sexy approach, there are a few items that are monetizable right out of the box:

- Portal Lite
- Identity verification
- Catalog management
- BI dashboards
- CWR registrations
- Accounting/GL sync

6.2 API Access (the Sexy Approach)



Once the kernel is live, URP can expose a suite of secure, versioned APIs that allow partners to directly read/write into the system. This is the moment Riptide stops acting like a publisher and starts behaving like Stripe, Twilio, or Plaid, etc., but for music rights. An end-to-end rights and money operating system that other companies build on top of.

Mark, you will recognize this model since it is akin to what Acme was working towards with its Avalanche subnet. It is a similar line of thinking and similar value prop, which is very big and super scalable.

Once APIs exist, the ecosystem no longer grows linearly (one catalog at a time). It grows exponentially, because partners are integrating software, instead relying on manual integrations or human maintenance.

6.3 Available APIs in the URP Suite

Identity API

Entities, roles, PRO IDs, ISNI/IPI mapping, creator relationships, merge events.

Agreement API

Canonical splits, terms, territories, amendments, version history.

Catalog API

Works, recordings, contributors, metadata lineage, remix relationships.

CWR API

Registration submission, status tracking, error queues, ingestion of PRO corrections.

Statement Ingestion API

Usage data, revenue events, partner metadata, recoupment fields.

BI / Analytics API

Territory insights, earnings trends, catalog performance, partner dashboards.

Payout API

Balances, pending payouts, schedules, GL entry preparation.

6.4 Why APIs Are the Most Valuable Part of the Business

A.) Everyone in the industry wants automation but no one has the rails.

Labels, publishers, UGC platforms, marketplaces, distributors, AI companies – everyone struggles to normalize rights, metadata, and payouts.

APIs let them route that complexity into URP instead of building it themselves, saving them time and money.

B.) APIs turn URP into critical infrastructure.

When partners automate contract ingestion, catalog updates, or usage ingestion through your APIs:

- They cannot easily leave
- They embed URP into their own products
- They design their workflows around your data model

This is how Stripe, Shopify, and Plaid became household names.

C.) APIs are the highest-margin revenue stream in the entire strategy.

Unlike admin fees or human workflows:

- API calls cost almost nothing to service
- Revenue scales automatically
- Partners self-serve
- Usage grows as partners grow

APIs = infinite gross margin + network effects.

D.) APIs attract a broader ecosystem than publishing alone.

APIs bring in:

- Educational platforms (SongHub-like)
- AI audio companies (metadata, splits, lineage)
- UGC remix platforms (Hook-like)
- Catalog buyers doing due diligence
- Labels wanting real-time earnings visibility
- PROs and DSPs seeking structured reporting
- Fintech companies integrating payouts

Publishing admin is a beachhead and APIs are the final-boss platform.

E.) APIs are the beginning of a rights-data marketplace.

Once URP controls the canonical records for the below, URP becomes the single source of truth for rights and revenue across multiple companies:

- **Identity** (writers, artists, publishers, contributors)
- **Agreements** (splits, terms, rights, amendments)
- **Works & Recordings** (catalog, versions, samples, remixes)
- **Lineage** (derivatives, UGC relationships, ownership chains)
- **Earnings** (usage, royalties, advances, recoupment)
- **Payouts** (who gets what, when, and why)

This transforms URP from a publishing platform into something much more defensible: The Rights & Royalty Data Marketplace for the Music Industry.

6.5 Here is how it could work.

A.) Step 1 – APIs Create Structured, Interoperable Data

With APIs, every partner:

- Sends structured data *into* URP
- Consumes structured data *from* URP
- Automates their workflows around URP's data model

This adds up to URP becoming the de facto schema for:

- Identity
- Works
- Catalog
- Registrations
- Contracts
- Usage
- Royalties
- Payouts

This is the same move Stripe made with payment primitives and Plaid made with bank account primitives.

Stripe succeeded by exposing payment primitives as a simple API, which replaced decades of fragmented banking integrations, and powered the e-commerce era.

Similarly, Plaid turned bank-account data into a universal open-banking API. That move enabled fintech, apps, neobanks, etc., to build without managing integrations for thousands of banks.

URP aims to do exactly that for music rights and royalties: expose identity, rights metadata, catalog, usage, inventory, and payouts as a unified, accessible API. This gives the industry (labels, publishers, DSPs, remix platforms, AI-music startups) a foundation to build on, unlocking scale, interoperability, and liquidity.

When everyone integrates your schema, your schema becomes the industry standard.

B.) Step 2 – Standardization Enables Aggregation

On-platform, you will eventually have:

- Hundreds of thousands of creators (SongHub + Hook + Riptide)
- Millions of works
- Millions of recordings
- Complete agreement metadata
- Usage across partners and DSPs
- Remix and UGC lineage
- Royalty and earnings history
- Payout history

Once all this flows into the same kernel, something remarkable happens, the data stops being “company data” and becomes “infrastructure data.”

URP becomes the only place where the full life cycle can be observed:

Creator > Agreement > Work > Recording > Usage > Earnings > Payout

No PRO, no DSP, no publisher, no distributor, no label has that entire view. Only URP does. And that’s what forms the basis of a marketplace.

C.) Step 3 – Rights & Royalty Data as Products

With canonical data, URP can begin offering data products other companies pay for.

Here are the actual marketplace verticals that emerge:

1. Sync Licensing Marketplace

URP knows:

- Who owns what
- Who the collaborators are
- Who gets paid
- What the splits are
- Whether a work is registered
- Who represents rights
- The approval chain

Meaning:

- Sync buyers (ad agencies, games, TV, film) can search URP's metadata.
- URP can offer eligibility, availability, rights clearance status.
- URP can show "license-ready" works.

Revenue: Referral fees, transaction fees, or licensing marketplace revenue share.

2. AI Training Data Licensing

AI companies need:

- Clean metadata
- Canonical identities
- Copyright lineage
- Rightsholder lists
- Derivatives mapping

- Historical usage data
- Licensing visibility

URP has exactly that dataset.

URP could offer:

- Datasets for training
- Cleared catalog bundles
- Pay-per-query fees
- Usage-based licensing

Revenue: This becomes a future-proof revenue stream that can be shared with URP's customer base and partners.

3. Catalog Valuation Data Services

Anyone trying to buy/sell a catalog needs to know:

- Earnings history
- Splits
- Contributor lists
- CWR status
- Recoupment positions
- Remix/UGC motion
- Risk indicators
- Territory performance

A buyer could subscribe to URP valuation insights.

Revenue: per-catalog valuation fee, data access subscription.

4. Royalty Forecasting & Predictive Analytics

Once URP has multi-catalog usage and payout patterns, it becomes possible to offer:

- Earnings predictions
- Territory forecasting
- UGC virality signals
- Catalog health scoring
- Sync likelihood predictions

Revenue: BI subscription tiers, but also could be the backbone of an entire lending structure to advance assets with interest ahead of earnings.

5. Metadata-as-a-Service

DSPs, UGC platforms, AI apps, and new music startups often don't want to manage metadata complexity.

URP can sell:

- Identity verification
- Canonical work metadata
- Relationship/lineage info
- Verified splits

Revenue: per-query pricing, API access tiers.

D.) Step 4 — How Our Partners Connect

Once partners integrate with URP APIs, we hit the dream flywheel:

More partners > more data > better analytics > more partners.

For example:

- SongHub creators generate works: improves catalog
- Hook generates UGC: improves lineage data
- Revelator sends usage data: improves earnings predictions
- Switchchord ingestion: improves identity and agreement accuracy

The marketplace becomes *self-reinforcing*, because every new integration improves the data for every other integration.

This is the same strategy that powers Stripe, Plaid, and Shopify.

6.6 The Big Point

APIs enable URP to standardize rights and royalty data across partners. Once the kernel becomes the industry's canonical source of identity, agreements, works, lineage, earnings, and payouts, URP evolves naturally into a rights-data marketplace — powering sync, AI, UGC, licensing, catalog valuation, and the next generation of music-tech applications.

7. Why This Strategy Is Credible

In short, we are not asking investors to fund an idea, we are asking them to fund the formalization of an already working system that could evolve into a dominant, must-have tool in the industry.

A.) Riptide already operates as a mini-publishing SaaS.

Creators submit works > Riptide processes them > registrations go out > royalties come in > payouts happen > statements close. The core publishing loop already runs in production. No new behavior is being invented, we are merely upgrading the architecture.

B.) The platform has real partners and a forming ecosystem.

Revelator, Switchchord, SongHub, and Hook are not hypotheticals, they are active collaborators with clear roles in the kernel/app model. The ecosystem is already in motion – URP gives it a shared brain.

C.) The kernel design is aligned with the industry's hardest problems.

Identity, catalog, contracts, CWR, royalty ingestion, BI, payouts – this is the unsexy, high-friction plumbing that almost no one wants to build, but everyone needs. URP squarely targets that gap and turns it into a product.

D.) Execution risk is real but bounded and staged.

We are not proposing a rebuild of Music Reports or Kobalt. The plan is:

- coexist with FileMaker via an adapter,
- define schemas first,
- ship an end-to-end MVP slice,
- evolve to v1,
- then open APIs and marketplace layers.

It's a lean, modular, partner-powered kernel based on one of the best wireframes we could ask for, our Filemaker platform.

E.) The upside is asymmetric.

Downside: we end up with a far better, more scalable internal stack for Riptide's own publishing business and it stops there.

Upside: we turn that stack into the rights and royalty infrastructure for an entire ecosystem with SaaS, API, and rights-data marketplace economics.
